

STALE FRUIT DETECTION USING SWIN TRANSFORMER

Introduction:

This project focuses on developing an automated system for detecting stale fruits using the Swin Transformer model. Detecting fruit freshness is crucial for reducing food waste and ensuring quality control throughout the supply chain.

Problem Statement:

We address the significant food waste and quality control issues in the supply chain caused by the inability to quickly and accurately identify stale or spoiled fruits. Manual inspection, often performed by **vendors** and other stakeholders, is subjective, time-consuming, and inefficient, leading to economic losses and potential health risks. Automated systems face challenges due to the visual similarity between fresh and early-stage stale fruits, requiring advanced computer vision techniques.

Objectives:

- To accurately classify fruits as 'fresh' or 'stale' using image processing and deep learning.
- To explore and evaluate the performance of various Transformer-based architectures, including Vision Transformers (ViT) and hybrid CNN+Swin models, for robust feature extraction and classification.
- To leverage the Swin Transformer as our primary model due to its demonstrated efficiency and effectiveness for this task.
- To ultimately provide a reliable and efficient method for quality assessment of fruits.

Dataset and Preprocessing:

For this project, we utilized a dataset of Fresh and Stale images of fruits and vegetables containing images of various fruits labeled as 'fresh' or 'stale'.

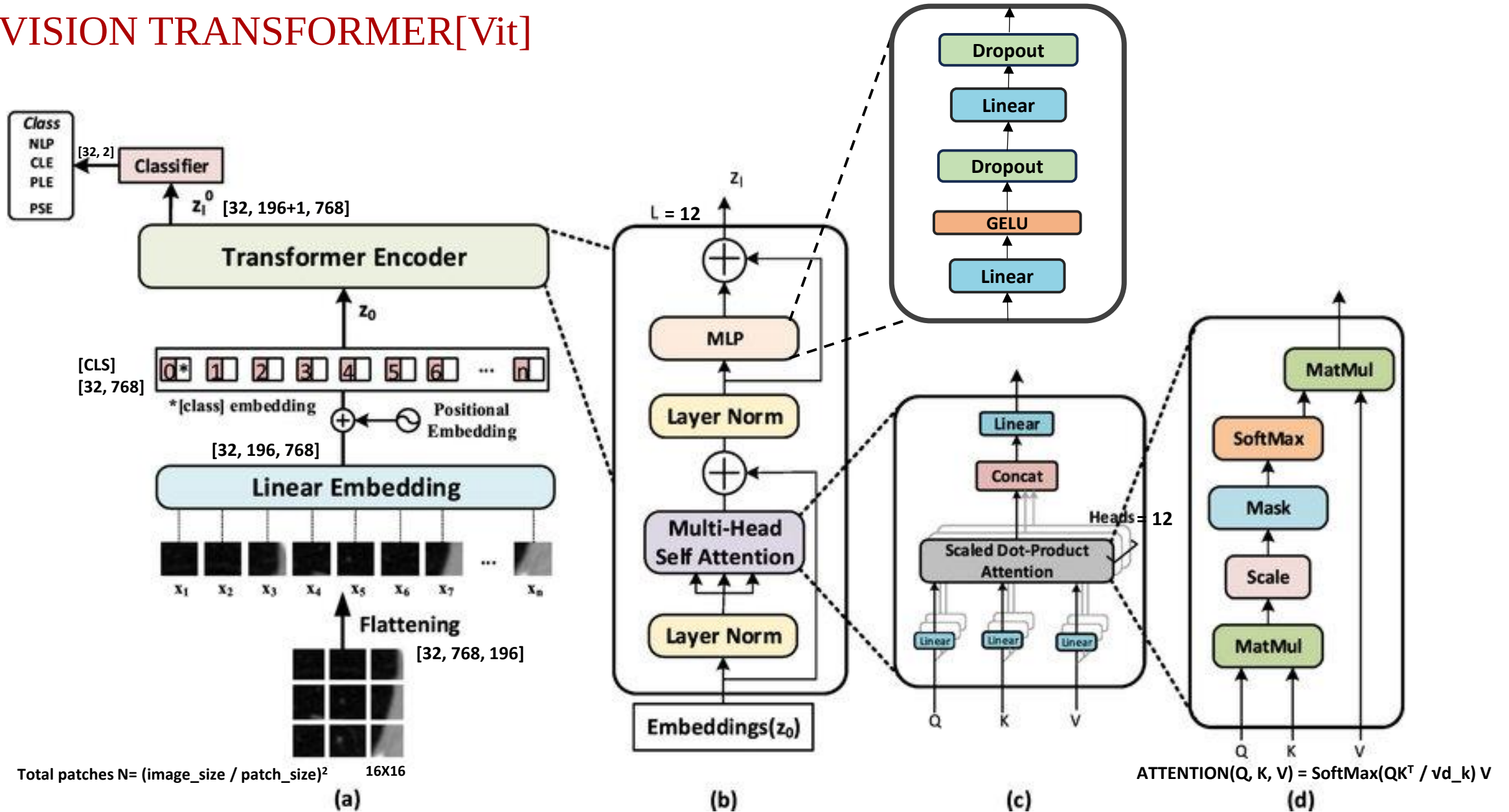
<https://www.kaggle.com/datasets/raghavrpotdar/fresh-and-stale-images-of-fruits-and-vegetables>.

Our dataset comprises approximately [14,682] images across [12] classes.

Preprocessing steps:

- Resizing images to a uniform resolution (224 X 224 pixels).
- Normalizing pixel values.
- Applying data augmentation techniques (e.g., random rotations, flips, color jitter) to enhance dataset diversity and prevent overfitting

VISION TRANSFORMER[ViT]



Patch Embedding Layer

```
class PatchEmbeddingLayer(nn.Module):
    def __init__(self, in_channels=3, patch_size=16, embedding_dim=768, img_size=224):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(in_channels, embedding_dim, kernel_size=patch_size, stride=patch_size)
        self.class_token = nn.Parameter(torch.randn(1, 1, embedding_dim))
        self.position_embeddings = nn.Parameter(torch.randn(1, self.num_patches + 1, embedding_dim))

    def forward(self, x):
        B = x.shape[0]
        x = self.proj(x).flatten(2).transpose(1, 2) # (B, N, C)
        cls_tokens = self.class_token.expand(B, -1, -1) # (B, 1, C)
        x = torch.cat((cls_tokens, x), dim=1) # (B, N+1, C)
        x = x + self.position_embeddings
        return x
```

Multi-Head Self-Attention Block

```
class MultiHeadSelfAttentionBlock(nn.Module):
    def __init__(self, embedding_dims=768, num_heads=12, attn_dropout=0.0):
        super().__init__()
        self.layernorm = nn.LayerNorm(embedding_dims)
        self.multiheadattention = nn.MultiheadAttention(
            embed_dim=embedding_dims,
            num_heads=num_heads,
            dropout=attn_dropout,
            batch_first=True
        )

    def forward(self, x):
        x_norm = self.layernorm(x)
        attn_output, _ = self.multiheadattention(x_norm, x_norm, x_norm, need_weights=False)
        return attn_output
```

MLP Block

```
class MachineLearningPerceptronBlock(nn.Module):
    def __init__(self, embedding_dims=768, mlp_size=3072, mlp_dropout=0.1):
        super().__init__()
        self.linear = nn.Sequential(
            nn.Linear(embedding_dims, mlp_size),
            nn.GELU(),
            nn.Dropout(mlp_dropout),
            nn.Linear(mlp_size, embedding_dims),
            nn.Dropout(mlp_dropout)
        )
        self.norm = nn.LayerNorm(embedding_dims)

    def forward(self, x):
        x_norm = self.norm(x)
        return self.linear(x_norm)
```

Transformer Block

```
class TransformerBlock(nn.Module):
    def __init__(self, embedding_dims=768, mlp_dropout=0.1, attn_dropout=0.0,
                 mlp_size=3072, num_heads=12):
        super().__init__()
        self.msa_block = MultiHeadSelfAttentionBlock(embedding_dims, num_heads,
                                                       attn_dropout)
        self.mlp_block = MachineLearningPerceptronBlock(embedding_dims, mlp_size,
                                                         mlp_dropout)

    def forward(self, x):
        x = x + self.msa_block(x)
        x = x + self.mlp_block(x)
        return x
```

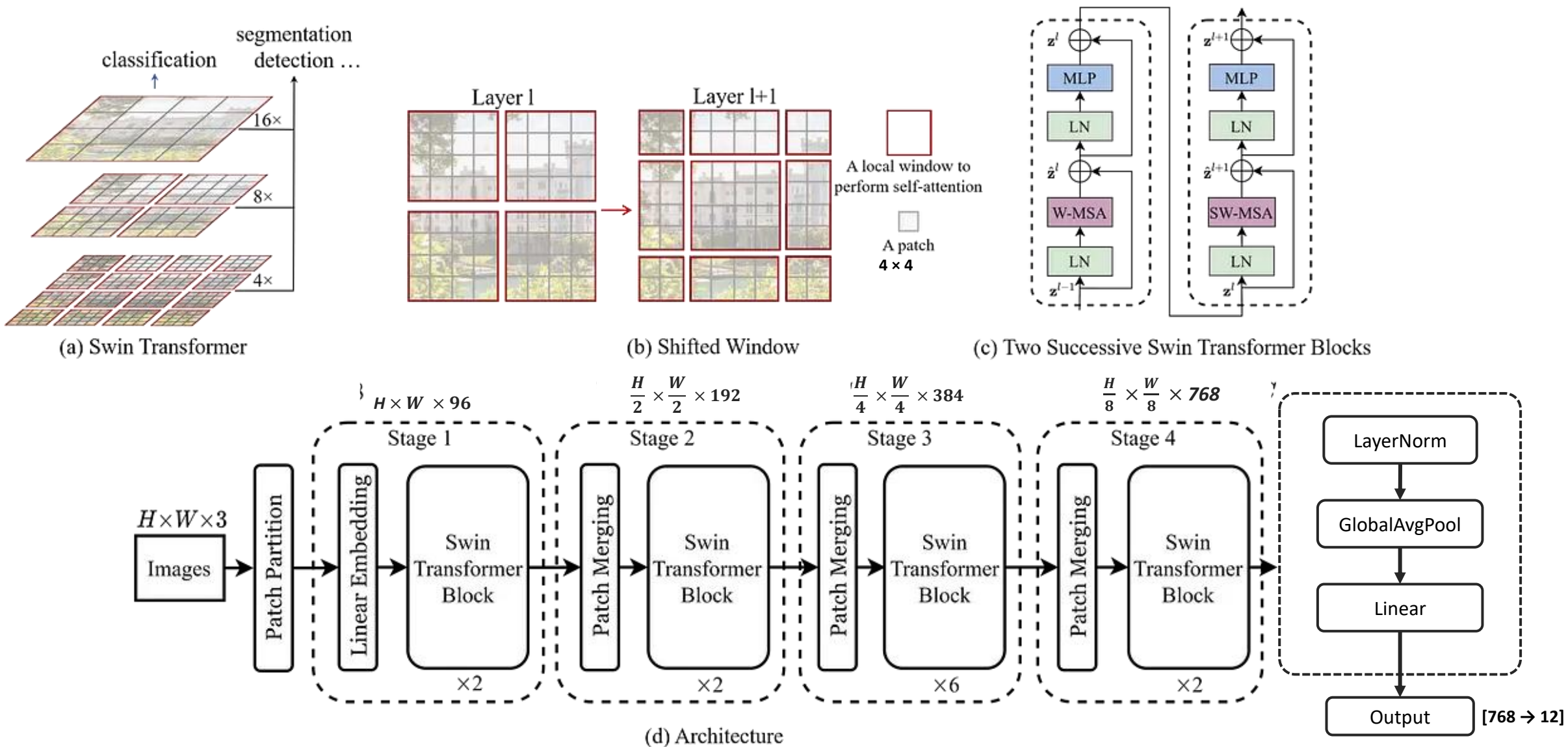
```

# Vision Transformer (ViT)
class VisionTransformer(nn.Module):
    def __init__(self, img_size=224, in_channels=3, patch_size=16, embedding_dims=768,
                  num_transformer_layers=12, mlp_dropout=0.1, attn_dropout=0.0, mlp_size=3072,
                  num_heads=12, num_classes=12):
        super().__init__()
        self.patch_embedding_layer = PatchEmbeddingLayer(in_channels, patch_size, embedding_dims, img_size)
        self.transformer_encoder = nn.Sequential(
            *[TransformerBlock(embedding_dims, mlp_dropout, attn_dropout, mlp_size, num_heads)
              for _ in range(num_transformer_layers)]
        )
        self.classifier = nn.Sequential(
            nn.LayerNorm(embedding_dims),
            nn.Linear(embedding_dims, num_classes)
        )

    def forward(self, x):
        x = self.patch_embedding_layer(x)
        x = self.transformer_encoder(x)
        x = x[:, 0] # CLS token
        return self.classifier(x)

```

Swin Transformer:



Patch Embedding

```
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=3, patch_size=4, embed_dim=96):
        super().__init__()
        self.proj = nn.Conv2d(in_channels, embed_dim, kernel_size=patch_size,
stride=patch_size)
```

```
    def forward(self, x):
        x = self.proj(x)
        x = x.flatten(2).transpose(1, 2) # B, N, C
        return x
```

Window Attention

```
class WindowAttention(nn.Module):
    def __init__(self, dim, window_size, num_heads):
        super().__init__()
        self.dim = dim
        self.window_size = window_size
        self.num_heads = num_heads

        self.scale = (dim // num_heads) ** -0.5
        self.qkv = nn.Linear(dim, dim * 3, bias=True)
        self.attn_drop = nn.Dropout(0.0)
        self.proj = nn.Linear(dim, dim)

    def forward(self, x):
        B_, N, C = x.shape
        qkv = self.qkv(x).reshape(B_, N, 3, self.num_heads, C //
self.num_heads)
        q, k, v = qkv.permute(2, 0, 3, 1, 4)

        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        out = (attn @ v).transpose(1, 2).reshape(B_, N, C)
        out = self.proj(out)
        return out
```

Window Partition / Reverse

```
def window_partition(x, window_size):
    B, H, W, C = x.shape
    x = x.view(B, H // window_size, window_size, W // window_size, window_size, C)
    windows = x.permute(0, 1, 3, 2, 4, 5).contiguous().view(-1, window_size, window_size, C)
    return windows
```

```
def window_reverse(windows, window_size, H, W):
    B = int(windows.shape[0] / (H * W / window_size / window_size))
    x = windows.view(B, H // window_size, W // window_size, window_size, window_size, -1)
    x = x.permute(0, 1, 3, 2, 4, 5).contiguous().view(B, H, W, -1)
    return x
```

Downsampling Layer (PatchMerging)

```
class PatchMerging(nn.Module):
    def __init__(self, input_resolution, dim):
        super().__init__()
        self.input_resolution = input_resolution
        self.dim = dim
        self.norm = nn.LayerNorm(4 * dim) # 4 * dim due to concatenation of 4 patches
        self.reduction = nn.Linear(4 * dim, 2 * dim, bias=False)
```

```
    def forward(self, x):
        H, W = self.input_resolution
        B, L, C = x.shape
        assert L == H * W, "Input feature dimensions do not match H*W"
        assert H % 2 == 0 and W % 2 == 0, f"Input resolution ({H}, {W}) must be
divisible by 2"
```

Reshape to (B, H/2, 2, W/2, 2, C) and permute to concatenate 4 neighboring patches

```
x = x.view(B, H // 2, 2, W // 2, 2, C)
x = x.permute(0, 1, 3, 2, 4, 5).contiguous() # (B, H/2, W/2, 2, 2, C)
x = x.view(B, (H // 2) * (W // 2), 4 * C) # Concatenate to 4*C
# Apply LayerNorm and Linear reduction
x = self.norm(x)
x = self.reduction(x) # (B, (H/2)*(W/2), 2*C)
return x
```



```
# Swin Transformer Block
```

```
class SwinBlock(nn.Module):
```

```
    def __init__(self, dim, input_resolution, num_heads, window_size=7, shift_size=0, mlp_ratio=4.0):
```

```
        super().__init__()
```

```
        self.dim = dim
```

```
        self.input_resolution = input_resolution
```

```
        self.num_heads = num_heads
```

```
        self.window_size = window_size
```

```
        self.shift_size = shift_size
```

```
        self.norm1 = nn.LayerNorm(dim)
```

```
        self.attn = WindowAttention(dim, window_size, num_heads)
```

```
        self.norm2 = nn.LayerNorm(dim)
```

```
        self.mlp = nn.Sequential(
```

```
            nn.Linear(dim, int(dim * mlp_ratio)),
```

```
            nn.GELU(),
```

```
            nn.Linear(int(dim * mlp_ratio), dim),
```

```
        )
```

```
    def forward(self, x):
```

```
        H, W = self.input_resolution
```

```
        B, L, C = x.shape
```

```
        assert L == H * W, "Input feature dimensions do not match H*W"
```

```
        shortcut = x
```

```
        x = self.norm1(x)
```

```
        x = x.view(B, H, W, C)
```

```
        # Cyclic shift
```

```
        if self.shift_size > 0:
```

```
            shifted_x = torch.roll(x, shifts=(-self.shift_size, -self.shift_size), dims=(1, 2))
```

```
        else:
```

```
            shifted_x = x
```

```
        # Partition windows
```

```
        x_windows = window_partition(shifted_x, self.window_size)
```

```
        # nW*B, window_size, window_size, C
```

```
        x_windows = x_windows.view(-1, self.window_size * self.window_size, C)
```

```
        # W-MSA/SW-MSA
```

```
        attn_windows = self.attn(x_windows)
```

```
        # Merge windows
```

```
        attn_windows = attn_windows.view(-1, self.window_size, self.window_size, C)
```

```
        shifted_x = window_reverse(attn_windows, self.window_size, H, W)
```

```
        # Reverse cyclic shift
```

```
        if self.shift_size > 0:
```

```
            x = torch.roll(shifted_x, shifts=(self.shift_size, self.shift_size), dims=(1, 2))
```

```
        else:
```

```
            x = shifted_x
```

```
        x = x.view(B, H * W, C)
```

```
        # FFN
```

```
        x = shortcut + x
```

```
        x = x + self.mlp(self.norm2(x))
```

```
        return x
```

```
# Swin Transformer Stage
```

```
class Stage(nn.Module):
    def __init__(self, dim, input_resolution, depth, num_heads, window_size,
downsample=False):
        super().__init__()
        self.blocks = nn.ModuleList([
            SwinBlock(
                dim=dim,
                input_resolution=input_resolution,
                num_heads=num_heads,
                window_size=window_size,
                shift_size=0 if (i % 2 == 0) else window_size // 2,
            ) for i in range(depth)
        ])
        self.downsample = PatchMerging(input_resolution, dim) if downsample else None

    def forward(self, x):
        for blk in self.blocks:
            x = blk(x)
        if self.downsample is not None:
            x = self.downsample(x)
        return x
```

```
# Swin Transformer
```

```
class SwinTransformer(nn.Module):
    def __init__(self, img_size=224, patch_size=4, in_channels=3, num_classes=12,
embed_dim=96, depths=[2, 2, 6, 2], num_heads=[3, 6, 12, 24], window_size=7):
        super().__init__()
        self.num_classes = num_classes
        self.num_layers = len(depths)
        self.embed_dim = embed_dim
        self.patch_embed = PatchEmbedding(in_channels, patch_size, embed_dim)
        self.pos_drop = nn.Dropout(p=0.0)
```

```
# Swin Transformer stages
```

```
self.layers = nn.ModuleList()
patches_resolution = img_size // patch_size
for i_layer in range(self.num_layers):
    stage = Stage(
        dim=embed_dim * (2 ** i_layer),
        input_resolution=(patches_resolution // (2 ** i_layer), patches_resolution // (2 ** i_layer)),
        depth=depths[i_layer],
        num_heads=num_heads[i_layer],
        window_size=window_size,
        downsample=(i_layer < self.num_layers - 1)
        # Downsample except for the last stage
    )
    self.layers.append(stage)

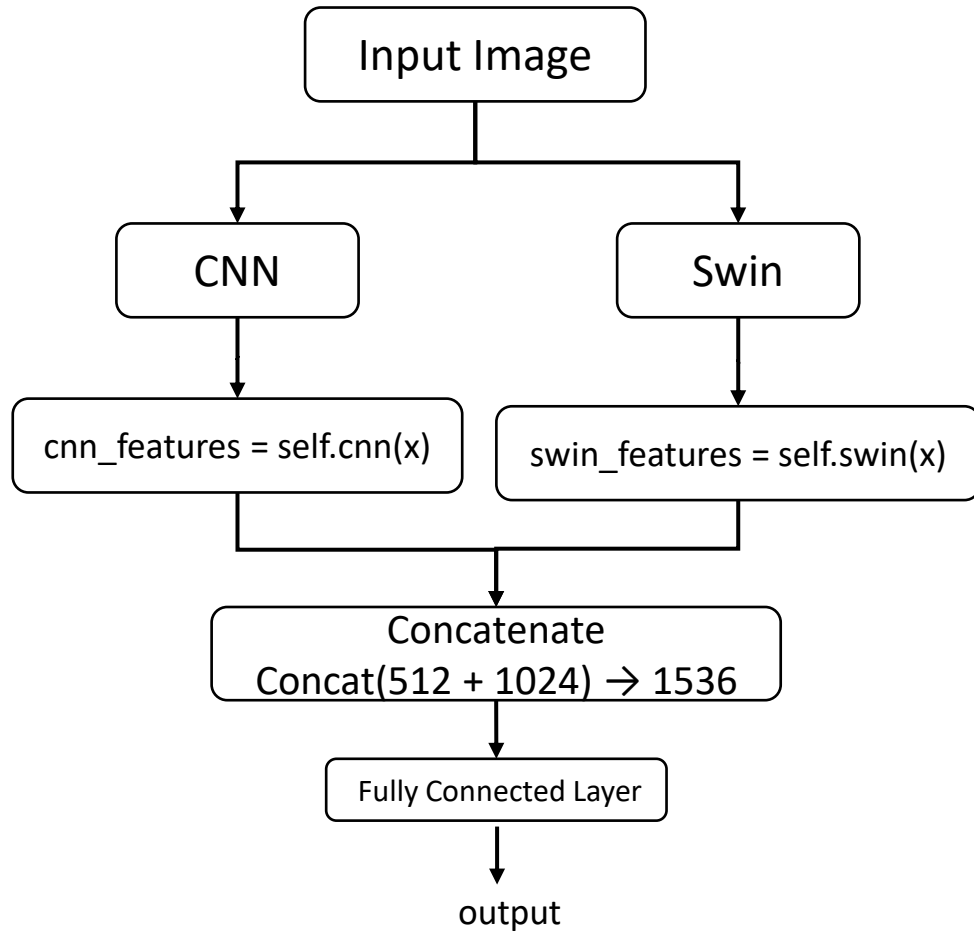
self.norm = nn.LayerNorm(embed_dim * (2 ** (self.num_layers - 1)))
self.avgpool = nn.AdaptiveAvgPool1d(1)
self.head = nn.Linear(embed_dim * (2 ** (self.num_layers - 1)), num_classes)
```

```
def forward(self, x):
    x = self.patch_embed(x)
    x = self.pos_drop(x)

    for layer in self.layers:
        x = layer(x)

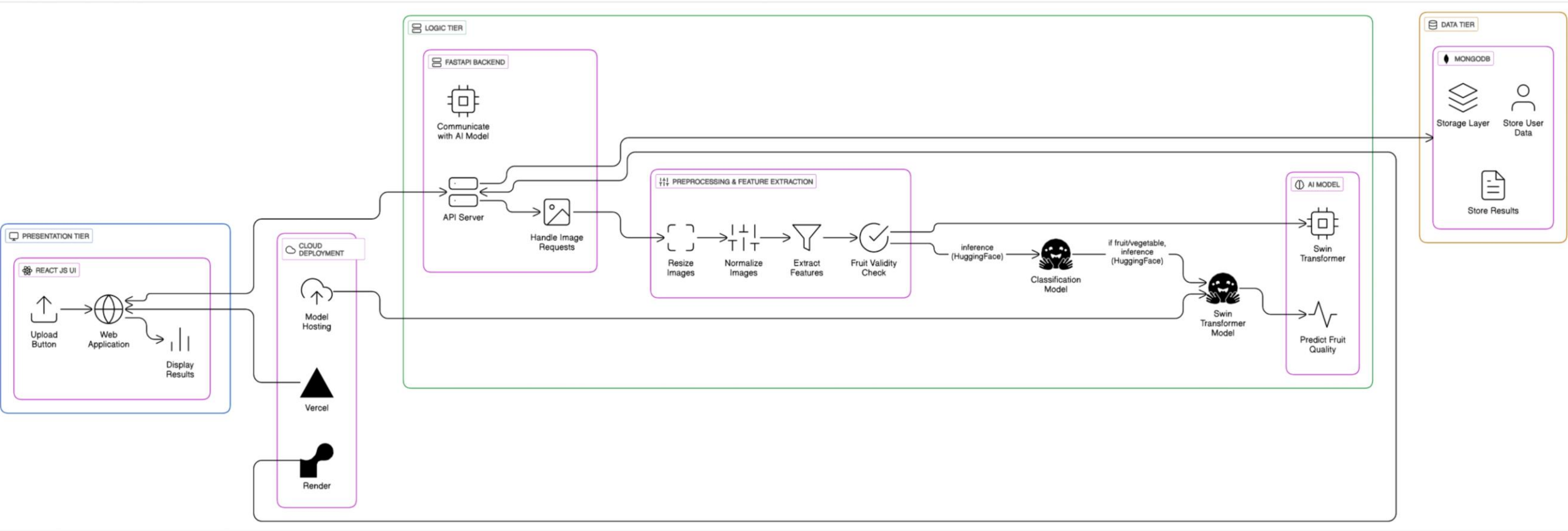
    x = self.norm(x)
    x = self.avgpool(x.transpose(1, 2)) # B, C, 1
    x = torch.flatten(x, 1)
    x = self.head(x)
    return x
```

Hybrid model (CNN+Swin):



We implemented a hybrid deep learning model that combines a **pretrained ResNet-18** and a **pretrained Swin Transformer (Swin-B)** for fruit image classification. The ResNet-18 serves as the CNN backbone for capturing local spatial features, while the Swin Transformer captures long-range dependencies through its hierarchical self-attention mechanism. Feature vectors from both models are concatenated and passed through a fully connected layer for final classification. This fusion approach leverages the strengths of both architectures, improving accuracy on the fruit dataset.

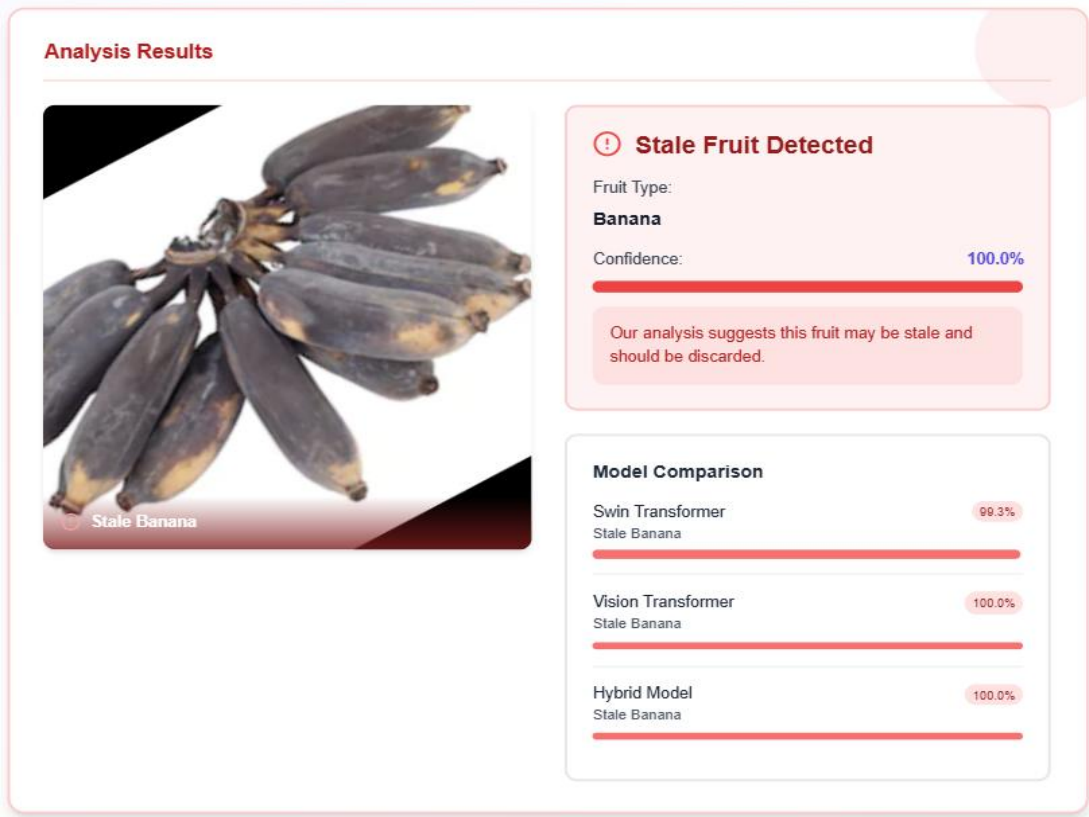
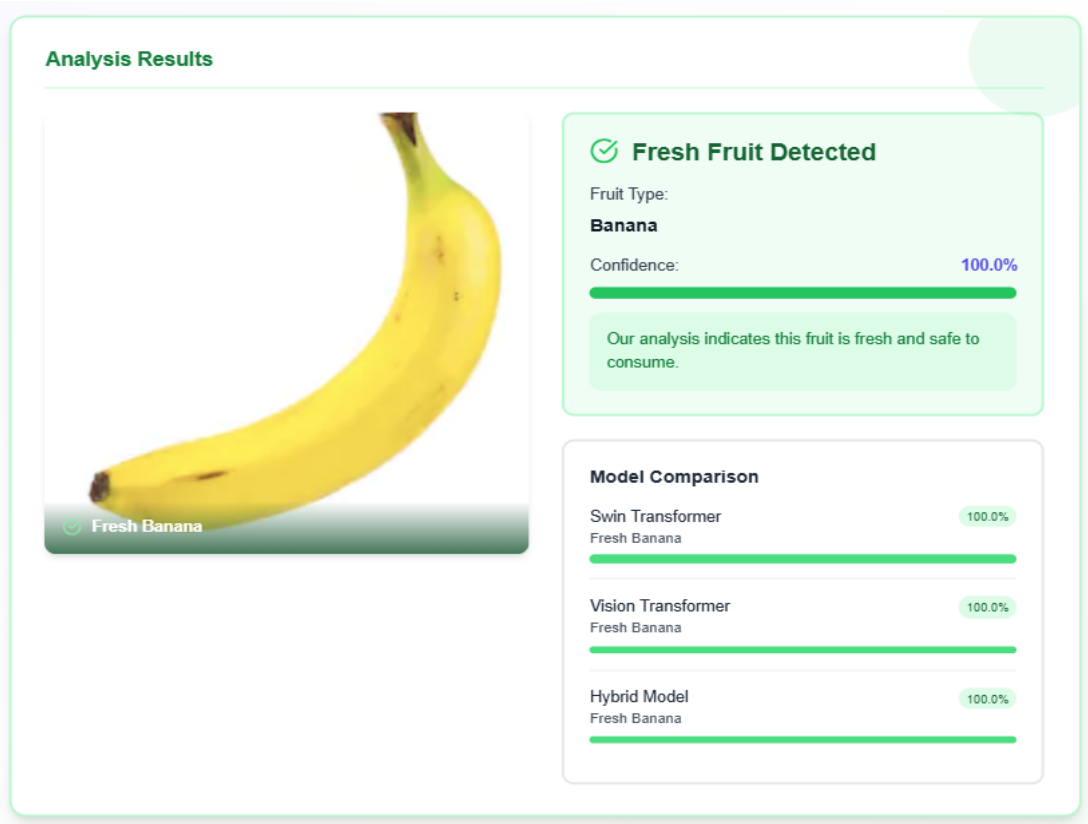
SYSTEM ARCHITECTURE



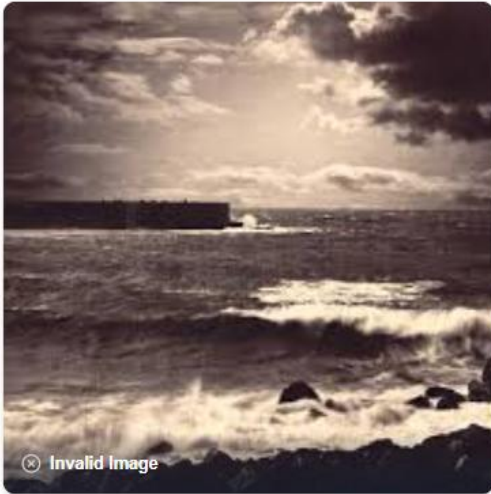
Training Details:

Parameter	Hybrid (CNN+Swin)	Vision Transformer (ViT)	Swin Transformer
Epochs	10	20	20
Batch Size	8	8	8
Learning Rate	1e-4 or 0.0001	1e-4 or 0.0001	1e-4 or 0.0001
Optimizer	Adam	Adam	Adam
Loss Function	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
Pre-training Used	resnet18	ViT-B/16 on IMAGENET_V1	--

Results:



Analysis Results



⊗ Invalid Image

The uploaded image could not be processed. Please try with a different image.

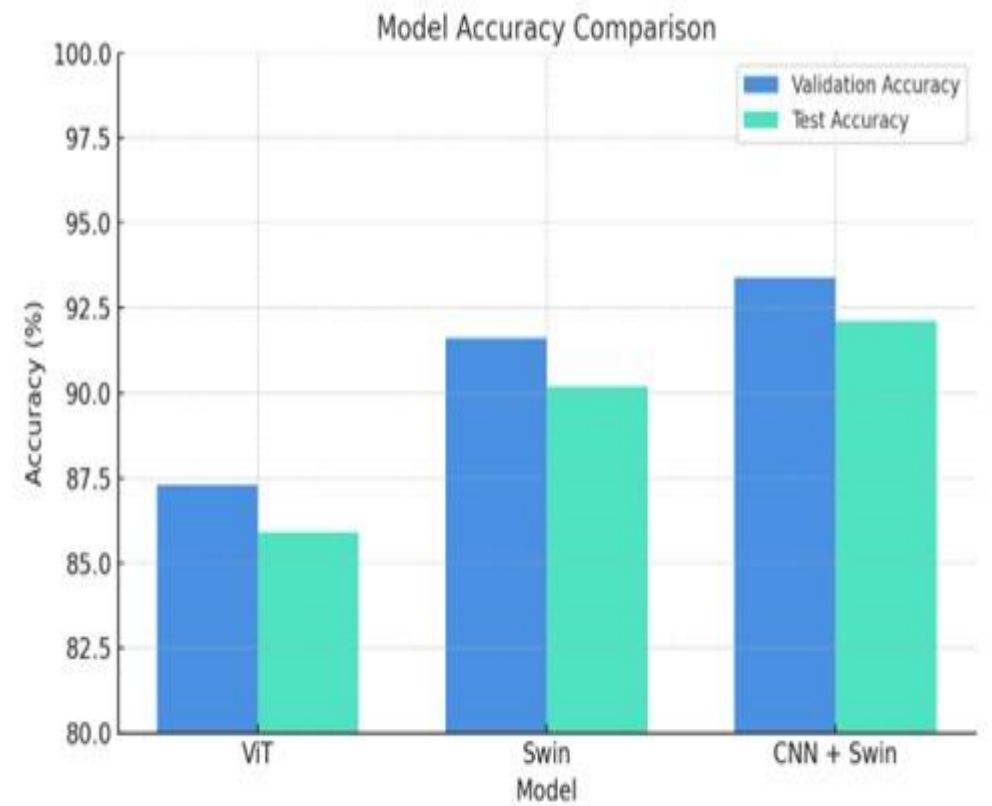
Possible issues:

- Image is not a fruit/vegetable
- Image is too blurry
- Unsupported file format
- Image is corrupted

Try Again

Upload a clear, well-lit photo of a single fruit or vegetable.

Upload New Image





Challenges

- Data Heterogeneity & Training Diversity
- AI Model Performance vs. Real-time Demands
- Model Interpretability & Trust
- Mitigating Overfitting in Complex Architectures

- Dynamic Data Augmentation Pipeline
- Optimized Architecture Selection & Asynchronous Backend
- Focus on Explainable AI (XAI) Principles
- Multi-Pronged Generalization Strategy



Solutions

Future Work

Our future work includes expanding to **multi-fruit detection and localization**, developing a more **granular freshness scoring** system, exploring **real-time video stream analysis**, and potentially integrating with **IoT devices**. We also aim to expand the dataset for better generalization and investigate **Explainable AI (XAI)** to understand model decisions better.

Conclusion and Q&A:

In conclusion, we have successfully developed an accurate and efficient system for stale fruit detection using the Swin Transformer, capable of significantly reducing food waste and improving quality control in the supply chain. We believe this project has substantial real-world impact.

We are now open for any questions you may have.

THANK YOU